

CWIP: Coupling With Interpolation Parallel Interface

Eric Quémerais¹, Bastien Andrieu¹, and Karmijn Hoogveld¹

¹ DMPE, ONERA, Université Paris-Saclay, 92320 Châtillon, France

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Christoph Junghans](#)

Reviewers:

- [@KennethKwabenaKenney](#)
- [@GuilloteauQ](#)
- [@MakisH](#)

Submitted: 03 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

CWIP is an open-source library dedicated to the parallel coupling of independent simulation codes for multi-physics research, a field also covered by tools such as MUI (Tang et al., 2015) and preCICE (Chourdakis et al., 2022). It is aimed at anyone wishing to conduct studies and research in this field. It provides MPI-based communication and fully distributed data exchange on non-coincident meshes, including on-the-fly spatial interpolation. CWIP stands out for its support of a wide variety of spatial interpolation families and a broad range of geometric element types. This extensive spatial capability allows users to leverage solver-specific numerical methods, including finite volumes, finite elements, discontinuous Galerkin, and spectral methods. Advanced users can further combine CWIP's geometric algorithms with solvers' internal interpolations through callbacks, improving accuracy across highly heterogeneous solvers.

CWIP supports a wide range of interface geometries, from linear, surface, and volumetric elements to higher-order elements with curved edges and faces. Performance-critical algorithms have been specifically optimized for handling coupling cases involving dynamic or adaptive meshes. Recent developments in the ParaDiGM library (Quémerais et al., 2026) now provide dynamic load and memory balancing for large-scale simulations. Ongoing work includes GPU acceleration of geometric algorithms (Cazalbou et al., 2024) and the development of temporal interpolation methods (Simon et al., 2025).

Statement of Need

Multi-physics problems can be approached either through monolithic methods — where all physical phenomena are solved simultaneously within a single solver — or through partitioned methods, where independent solvers exchange data across shared interfaces. Partitioned approaches are particularly attractive in high-performance computing (HPC) contexts, as they allow the reuse of existing specialized codes while maintaining their native parallel architecture.

Robust and efficient coupling tools are essential to enable data exchange between solvers operating on distinct meshes, geometries, and discretization schemes. CWIP was initially developed in 2009 to provide an open-source solution complementary to MPCCI (Joppich & Kürschner, 2006), whose black-box nature limited flexibility and transparency for multi-physics research.

To ensure a straightforward and efficient deployment of coupled simulations, an HPC coupling tool must abstract the complexity of parallelism by interacting solely with data local to each MPI rank. Furthermore, it must deliver high-performance spatial interpolation algorithms whose execution time remains negligible compared to a single solver iteration, thereby avoiding any penalty on the overall simulation wall-clock time. Alongside these, it must provide temporal coupling schemes—or at least the essential building blocks to implement them. While meeting these requirements is sufficient for standard use cases, the ability to implement specific spatial interpolation methods is critical to achieve a global accuracy order close to

the solvers' internal precision. This necessity was demonstrated by (Léger et al., 2012) for an isophysical configuration based on the linearized Euler equations: by exposing CWIPI's geometric algorithms to the solvers' internal interpolation capabilities via callbacks, and by employing the same Runge-Kutta scheme with coupling performed at the sub-iteration level, a global accuracy order of 3.5 was preserved when coupling a 4th-order finite difference code with a 4th-order discontinuous Galerkin solver.

State of the Field

Existing coupling frameworks differ significantly in scope, geometry support, and interpolation capabilities. MUI (Tang et al., 2015) primarily supports point clouds and focuses on general-purpose temporal and spatial interpolation. preCICE (Chourdakis et al., 2022) supports point clouds or meshes composed of planar triangular or quadrilateral elements, provides a broad ecosystem of adapters for established simulation codes, and integrates black-box temporal coupling algorithms. In contrast to these tools, OpenPALM (Duchaine et al., 2015) features a dedicated supervision process that orchestrates temporal coupling algorithms, which are defined beforehand by the user through the PrePALM graphical user interface. In this architecture, OpenPALM operates as a higher-level coupler, relying entirely on CWIPI as its underlying spatial interpolation and low-level communication engine. Although a comparative study of coupling tools within an HPC framework (Rubin, 2022) has been conducted, it is not exhaustive; it mentions other tools besides those cited above but only discusses CWIPI through its utilization via OpenPALM.

Regarding temporal coupling algorithms, and unlike black-box solutions such as preCICE, CWIPI does not offer any turn-key solutions; instead, it solely provides the building blocks to define the orchestration of exchanges. Consequently, the temporal coupling algorithm is defined by the user in a decentralized manner.

Regarding spatial interpolation of fields, a concept designated as “mapping” or “sampling” depending on the framework, CWIPI, due to its HPC nature, has oriented itself exclusively towards spatial interpolation methods based on local neighboring data. This contrasts notably with preCICE which, in addition to local interpolation methods, early on proposed a set of global RBF-type interpolation methods, involving all nodes of the interface mesh.

CWIPI offers greater versatility than the solutions mentioned in terms of geometric representation of interfaces. While preCICE's “mapping” or MUI's “sampling” rely mainly on point cloud-based interpolation, CWIPI supports a broad spectrum of interface representations. This includes linear, surface and volumetric elements, polygons and polyhedra, as well as higher-order elements with curved edges and faces. To exploit these geometries, CWIPI provides three families of interpolation methods based on an underlying geometric algorithm: one based on point clouds (similar to preCICE and MUI), one based on locating target points within the source mesh, and one based on the intersection of source and target meshes. For each of these, a default method is provided, supplemented by a callback interface allowing users to implement the interpolation method best suited to their specific context. If MUI also allows users to write a custom callback, this is limited to point clouds; conversely, CWIPI extends this capability to the three geometric families. Within a CWIPI callback, parallelism management is entirely transparent: the user works exclusively with local data, without having to manage remote communications, except in the specific case where they invoke internal functions of their own code requiring their intracommunicator. It should be noted that the family based on locating target points was the first to be implemented, which explains its privileged use in the vast majority of CWIPI applications.

Strategies for launching and communication differ significantly across frameworks. MUI and CWIPI favor an approach based on MPI MPMD (Multiple Program, Multiple Data), whereas preCICE opts for a decoupled approach where each code is launched via its own independent MPI command (MPI Port), with the connection between processes established a posteriori.

92 Although CWIPI offers a TCP/IP communication mode similar to preCICE, it is rarely used,
93 as the requirements of HPC users almost exclusively necessitate the use of MPI.

94 CWIPI stands out for its high flexibility in distributing codes across MPI ranks. Unlike
95 approaches that mandate disjoint communicators, CWIPI allows a code to be placed on any
96 rank. Consequently, the intersection of the MPI intra-communicators of different codes is
97 not necessarily empty. In the limiting case where codes share all their ranks, their intra-
98 communicators become duplications of the same communicator. This architecture offers total
99 freedom: the same coupling case can be handled with disjoint intra-communicators or fully
100 overlapping ones, without any assumptions regarding the location of the coupling interface. In
101 practice, although this capability is available in C and Fortran, it will mainly be used in Python,
102 since the flexibility of this language makes it easy to load the module for a given coupled code
103 depending on the MPI rank number, and thus to distribute the codes as desired. This capability
104 therefore makes it possible to adjust this distribution in order to improve load balancing across
105 the whole coupled simulation. CWIPI does not provide a capability for searching for an optimal
106 distribution, but this could constitute a future research direction.

107 Software Design

108 Since version 1.0, CWIPI is built on top of the ParaDiGM library (Qu  merais et al., 2026),
109 originally created specifically to meet the needs of CWIPI. ParaDiGM provides the low-level
110 geometric algorithms and parallel data structures. More broadly, ParaDiGM was designed to
111 mutualize parallel geometric algorithms that share common requirements across other contexts,
112 such as dynamic mesh adaptation and, more generally, any geometric co-processing tasks
113 performed by solvers during runtime.

114 CWIPI provides three APIs — C/C++, Fortran, and Python/NumPy — making it compatible
115 with a wide range of scientific codes. The integration of CWIPI into an existing solver is
116 designed to be minimally intrusive: it does not require modifying the code structure, and the
117 API relies solely on simple arrays, keeping the coupling layer lightweight and straightforward to
118 adopt.

119 To support various discretization schemes, CWIPI's three interpolation families provide distinct
120 default approaches and data structures for custom implementations. The first family, based on
121 searching for the k-nearest source nodes for each target point, operates on nodes, cell centers,
122 or user-defined points for both source and target locations. Its default method relies on a
123 least squares approach, and its callback interface exposes, for each target point, the local data
124 containing the k-nearest source points, their distances, and the associated source field values.
125 The second family focuses on the location of target points within the source mesh. It maps
126 source fields defined at nodes or cell centers onto target fields located at nodes, cell centers,
127 or user-defined points. By default, it uses the Lagrange basis functions of the source elements;
128 for custom interpolations, the callback interface provides the data local to each source element,
129 specifically the target points it contains (or their projections) and the source element fields.
130 The third family is based on the intersection of source and target meshes, where both fields
131 are strictly located at cell centers. Its default method performs a weighting by the volumes of
132 the intersecting polyhedra, and the callback interface supplies each target element with its
133 exact intersection volumes and the fields defined on the intersecting source polyhedra.

134 At each coupling step, the execution sequence typically proceeds as follows: first, the geometric
135 algorithm corresponding to the selected interpolation family is invoked; second, the physical
136 fields are exchanged across the communication graph established by the preceding geometric
137 step; and finally, the fields are interpolated onto the target mesh. However, if the chosen family
138 is based on the location of target points within the source mesh, the field interpolation is
139 performed directly by the source code, which inverts the order of the last two steps. Furthermore,
140 in cases where the mesh interface remains fixed throughout the simulation, the geometric
141 algorithm is invoked only once during the initialization phase, and its results are subsequently

142 stored in memory for all remaining coupling steps.

143 Within this coupling communicator, the geometric algorithms linked to the interpolation
144 families generate optimized point-to-point communication graphs between the processes of the
145 coupled codes. Each geometric algorithm produces its own communication graph. This graph
146 also depends on the location of the fields' degrees of freedom (nodes, cell centers, etc.). Since
147 the location of the degrees of freedom and the choice of interpolation family are properties
148 specific to each field, CWIPI is capable of managing several distinct communication graphs
149 between the codes for a single coupling instance.

150 A major design challenge lies in the **performance of geometric algorithms**, especially when the
151 coupling interface moves during the simulation or when solvers employ dynamic mesh adaptation.
152 In the 0.x branch of CWIPI, algorithms relied on the FVM library from Code_Saturne
153 (Archambeau et al., 2004), which later evolved into PLE, a lightweight coupling library
154 (Fournier, 2020). New algorithms (Andrieu et al., 2026) developed within ParaDiGM provide
155 **dynamic load and memory balancing** at each coupling step, which is critical because coupling
156 problems are inherently unbalanced across MPI processes. These developments bring significant
157 performance gains for large-scale and complex simulations.

158 Future Directions

159 Two main research directions are currently pursued. The first concerns the **GPU porting** of the
160 most computationally expensive geometric algorithms, for which new algorithms have recently
161 been designed (Cazalbou et al., 2024) and will be integrated in an upcoming release. The
162 second focuses on the development of **temporal interpolation methods** (François & Massot,
163 2023; Simon et al., 2025), complementary to those provided by existing frameworks such as
164 preCICE, aiming to improve accuracy and flexibility for dynamic interfaces and heterogeneous
165 solvers.

166 Research Impact Statement

167 From a scientific perspective, CWIPI is primarily used within the aeronautics and aerospace
168 community from which it originated. The library is integrated into several codes developed
169 at ONERA, including elsA (Cambier et al., 2013) for computational fluid dynamics, CEDRE
170 (Refloch et al., 2011) for multiphysics simulations, Z-set (Garaud et al., 2019) for structural
171 and materials mechanics, MoDeTheC (Dellinger et al., 2024) for thermal degradation of
172 materials, and SPACE (Delorme et al., 2005) for acoustic propagation. Through these tools,
173 CWIPI enables advanced multiphysics simulations in aerothermodynamics (Errera et al., 2019),
174 aeroacoustics (Langenais et al., 2019; Moratilla-Vega et al., 2022), magnetohydrodynamics
175 (Rocamora et al., 2025), electric arc propagation, and fire safety (Dellinger et al., 2023).

176 CWIPI is also employed as both an internal and external coupling library in leading combustion
177 simulation environments such as YALES2 (Moureau et al., 2011) and AVBP (Schonfeld &
178 Rudgyard, 1999). A notable HPC application (Dombard et al., 2018) combining CWIPI with
179 AVBP on a complex geometry demonstrated the robustness and scalability of the coupling
180 approach on massively parallel architectures. Its capabilities were highlighted as early as 2017
181 (Duchaine et al., 2017) and confirmed by an invitation to the ExCALIBUR workshop (Andrieu
182 & Quémerais, 2021). Recent work in aeroelasticity with YALES2 (Fabbri et al., 2023) further
183 demonstrates its versatility. CWIPI's integration into the software stack of national HPC
184 facilities such as the CCRT confirms its recognition at a national level.

185 Beyond its original ecosystem, the open-source community has adopted CWIPI through GitHub-
186 based interfacing projects with OpenFOAM (Moratilla, 2022; nd-92, 2025; Weller et al.,
187 1998) and Nektar++ (Cantwell et al., 2015), enabling applications in aeroacoustics and data
188 assimilation (Valero, 2026). Finally, CWIPI serves as the interpolation engine of the OpenPALM

coupler (Duchaine et al., 2015), illustrating its central role in the broader multiphysics and HPC coupling ecosystem.

AI Usage Disclosure

Generative AI tools were used to assist with the writing of this paper: translation, formatting, and improving the fluency of the text. No AI was used in the development of the CWIPI software. All scientific content was written and validated by the authors.

Acknowledgements

The authors would like to thank the following people for their contributions to the development, testing, and dissemination of the CWIPI library within the community:

Romain Casta¹, Nicolas Dellinger², Florent Duchaine¹, Bastien Frisulli², Jean-Didier Garaud², Thomas Hennion², Stéphanie Lala², Xavier Lamboley^{1,2}, Bruno Maugars², Thierry Morel¹, Christophe Peyret²

¹ CERFACS — ² ONERA

The authors also wish to thank BPI France, the Directorate General for Civil Aviation (DGAC), and the General Scientific Directorate of ONERA for their financial support.

Author Contributions

The contributions to this software are listed according to the CRediT taxonomy:

- **Eric Quémerais:** Conceptualization, Methodology, Software, Validation, Writing – Original Review & Editing, Project Administration, Funding Acquisition, Supervision
- **Bastien Andrieu:** Software, Validation, Writing – Original Draft
- **Karmijn Hoogveld:** Software, Validation, Writing – Original Draft

References

- Andrieu, B., Maugars, B., & Quémerais, E. (2026). Dynamic load-balanced point location algorithm for data mapping. *Comptes Rendus. Mécanique*, 354(G1), 53–70. <https://doi.org/10.5802/crmeca.335>
- Andrieu, B., & Quémerais, E. (2021, December). The CWIPI library. *ExCALIBUR ISE Code Coupling Workshop 1*. https://excalibur-coupling.github.io/workshop1_content/slides/B-Andrieu_ISE_CWIPI.pdf
- Archambeau, F., Méchitoua, N., & Sakiz, M. (2004). Code_Saturne: A finite volume code for the computation of turbulent incompressible flows. *International Journal on Finite Volumes*. <https://hal.archives-ouvertes.fr/hal-01115371/document>
- Cambier, L., Heib, S., & Plot, S. (2013). The onera elsA CFD software: Input from research and feedback from industry. *Mechanics & Industry*, 14, 159–174. <https://doi.org/10.1051/meca/2013056>
- Cantwell, C. D., Moxey, D., Comerford, A., Bolis, A., Rocco, G., Mengaldo, G., De Grazia, D., Yakovlev, S., Lombard, J.-E., Ekelschot, D., Jordi, B., Xu, H., Mohamied, Y., Eskilsson, C., Nelson, B., Vos, P., Biotto, C., Kirby, R. M., & Sherwin, S. J. (2015). Nektar++: An open-source spectral/hp element framework. *Computer Physics Communications*, 192, 205–219. <https://doi.org/10.1016/j.cpc.2015.02.008>

- 228 Cazalbou, R., Duchaine, F., Quémerais, E., Andrieu, B., Staffebach, G., & Maugars, B. (2024).
 229 *Hybrid multi-GPU distributed octrees construction for massively parallel code coupling*
 230 *applications*. <https://doi.org/10.1145/3659914.3659928>
- 231 Chourdakis, G., Davis, K., Rodenberg, B., Schulte, M., Simonis, F., Uekermann, B., Abrams,
 232 G., Bungartz, H., Cheung Yau, L., Desai, I., Eder, K., Hertrich, R., Lindner, F., Rusch,
 233 A., Sashko, D., Schneider, D., Totounferoush, A., Volland, D., Vollmer, P., & Koseomur,
 234 O. (2022). preCICE v2: A sustainable and user-friendly coupling library. *Open Research*
 235 *Europe*, 2, 51. <https://doi.org/10.12688/openreseurope.14445.2>
- 236 Dellinger, N., Donjat, D., Laroche, E., & Reulet, P. (2023). Experimental and numerical
 237 modelling of the interaction between a turbulent premixed propane/air flame and a
 238 composite flat plate. *Fire Safety Journal*, 141, 103959. <https://doi.org/10.1016/j.firesaf.2023.103959>
- 240 Dellinger, N., Leplat, G., Huchette, C., Biasi, V., & Feyel, F. (2024). Numerical modeling and
 241 experimental validation of heat and mass transfer within decomposing carbon fibers/epoxy
 242 resin composite laminates. *International Journal of Thermal Sciences*, 201, 109040.
 243 <https://doi.org/10.1016/j.ijthermalsci.2024.109040>
- 244 Delorme, Ph., Mazet, P.-A., Peyret, C., & Ventribout, Y. (2005). Computational aeroacoustics
 245 applications based on a discontinuous galerkin method. *Comptes Rendus Mécanique*,
 246 333(9), 676–682. <https://doi.org/10.1016/j.crme.2005.07.007>
- 247 Dombard, J., Duchaine, F., Gicquel, L. Y. M., Staffebach, G., Buffaz, N., & Trébinjac, I.
 248 (2018). *Large eddy simulations in a transonic centrifugal compressor*. V02BT44A030.
 249 <https://doi.org/10.1115/GT2018-77023>
- 250 Duchaine, F., Berger, S., Staffebach, G., & Gicquel, L. (2017). Partitioned high performance
 251 code coupling applied to CFD. In E. Di Napoli, M. A. Hermanns, H. Iliev, A. Lintermann,
 252 & A. Peyser (Eds.), *High-performance scientific computing* (Vol. 10164). Springer.
 253 https://doi.org/10.1007/978-3-319-53862-4_1
- 254 Duchaine, F., Jauré, S., Poitou, D., Quémerais, E., Staffebach, G., Morel, T., & Gicquel,
 255 L. (2015). Analysis of high performance conjugate heat transfer with the OpenPALM
 256 coupler. *Computational Science & Discovery*, 8(1), 015003. <https://doi.org/10.1088/1749-4699/8/1/015003>
- 258 Errera, M.-P., Moretti, R., Salem, R., Bachelier, Y., Arrivé, T., & Nguyen, M. (2019). A
 259 single stable scheme for steady conjugate heat transfer problems. *Journal of Computational*
 260 *Physics*, 394, 491–502. <https://doi.org/10.1016/j.jcp.2019.05.036>
- 261 Fabbri, T., Balarac, G., Moureau, V., & Benard, P. (2023). Design of a high fidelity fluid-
 262 structure interaction solver using LES on unstructured grid. *Computers & Fluids*, 265,
 263 105963. <https://doi.org/10.1016/j.compfluid.2023.105963>
- 264 Fournier, Y. (2020). Massively parallel location and exchange tools for unstructured meshes.
 265 *International Journal of Computational Fluid Dynamics*, 34(7-8), 1–20. <https://doi.org/10.1080/10618562.2020.1810676>
- 267 François, L., & Massot, M. (2023). Multistep interface coupling for high-order adaptive
 268 black-box multiphysics simulations. *Proceedings of the 10th International Conference*
 269 *on Computational Methods for Coupled Problems in Science and Engineering*. <https://doi.org/10.23967/c.coupled.2023.027>
- 271 Garaud, JD., Rannou, J., Bovet, C., Feld-Payet, S., Chiaruttini, V., Marchand, B., Lacourt,
 272 L., Yastrebov, V. A., Osipov, N., & Quilici, S. (2019, May). Z-set -suite logicielle pour la
 273 simulation des matériaux et structures. *14ème Colloque National en Calcul de Structures*
 274 *(CSMA 2019)*. <https://hal.science/hal-02115875>
- 275 Joppich, W., & Kürschner, M. (2006). MpCCI – a tool for the simulation of coupled

- 276 applications. *Concurrency and Computation: Practice and Experience*, 18(2), 183–192.
 277 <https://doi.org/10.1002/cpe.913>
- 278 Langenais, A., Vuillot, F., Troyes, J., & Bailly, C. (2019). Accurate simulation of the noise
 279 generated by a hot supersonic jet including turbulence tripping and nonlinear acoustic
 280 propagation. *Physics of Fluids*, 31(1), 016105. <https://doi.org/10.1063/1.5050905>
- 281 Léger, R., Peyret, C., & Piperno, S. (2012). Coupled discontinuous galerkin/finite difference
 282 solver on hybrid meshes for computational aeroacoustics. *AIAA Journal*, 50, 338–349.
 283 <https://doi.org/10.2514/1.J051110>
- 284 Moratilla, M. (2022). *Coupling-API: Open source API to connect OpenFOAM to Nektar++ for*
 285 *aeroacoustic simulations*. GitHub repository. <https://github.com/mimove/Coupling-API>
- 286 Moratilla-Vega, M. A., Angelino, M., Xia, H., & Page, G. J. (2022). An open-source coupled
 287 method for aeroacoustics modelling. *Computer Physics Communications*, 278, 108420.
 288 <https://doi.org/10.1016/j.cpc.2022.108420>
- 289 Moureau, V., Domingo, P., & Vervisch, L. (2011). Design of a massively parallel CFD
 290 code for complex geometries. *Comptes Rendus Mécanique*, 339(2), 141–148. <https://doi.org/10.1016/j.crme.2010.12.001>
- 292 nd-92. (2025). *cwipiFoam: Coupling between compressible OpenFOAM solvers and Nektar++*
 293 *AcousticSolver*. GitHub repository. <https://github.com/nd-92/cwipiFoam>
- 294 Quémerais, E., Andrieu, B., Maugars, B., & Hoogveld, K. (2026). *ParaDiGM*. <https://github.com/onera/paradigm>.
- 296 Refloch, A., Courbet, B., Murrone, A., Villedieu, P., Laurent, C., Gilbank, P., Troyes, J., Tessé,
 297 L., Chaineray, G., Dargaud, J. B., Quémerais, E., & Vuillot, F. (2011). CEDRE Software.
 298 *Aerospace Lab*, 2, p. 1–10. <https://hal.science/hal-01182463>
- 299 Rocamora, A., Labaune, J., Laux, C. O., & Tholin, F. (2025). Numerical study of plasma-
 300 assisted ignition and stabilisation of a supersonic hydrogen–air combustion in LAERTE
 301 facility by a quasi-DC gliding arc. *Plasma Sources Science and Technology*, 34(10), 105002.
 302 <https://doi.org/10.1088/1361-6595/ae00ee>
- 303 Rubin, P. (2022). *Comparison of code coupling libraries for high performance multi-physics*
 304 *simulation* (DL Technical Reports DL-TR-2022-001). Science; Technology Facilities Council.
 305 <https://doi.org/10.5286/dltr.2022001>
- 306 Schonfeld, T., & Rudgyard, M. (1999). Steady and unsteady flow simulations using the hybrid
 307 flow solver AVBP. *AIAA Journal*, 37(11), 1378–1385. <https://doi.org/10.2514/2.636>
- 308 Simon, A., François, L., & Massot, M. (2025). High-order multistep coupling: Convergence,
 309 stability and PDE application. *Comptes Rendus. Mécanique*, 353, 1159–1184. <https://doi.org/10.5802/crmeca.333>
- 311 Tang, Y.-H., Kudo, S., Bian, X., Li, Z., & Karniadakis, G. E. (2015). Multiscale universal
 312 interface: A concurrent framework for coupling heterogeneous solvers. *Journal of*
 313 *Computational Physics*, 297, 13–31. <https://doi.org/10.1016/j.jcp.2015.05.004>
- 314 Valero, M. (2026). *CONES: Coupling OpenFOAM with Numerical EnvironmentS*. GitHub
 315 repository. <https://github.com/MiguelMValero/CONES>
- 316 Weller, H. G., Tabor, G., Jasak, H., & Fureby, C. (1998). A tensorial approach to computational
 317 continuum mechanics using object-oriented techniques. *Computers in Physics*, 12(6),
 318 620–631. <https://doi.org/10.1063/1.168744>